

Step 1: Exploratory Data Analysis & Data Manipulation.

```
# Importing essential libraries for data analysis and manipulation..
```

```
import pandas as pd
import numpy as np
```

```
import pandas as pd
```

```
# Loading the dataset..
df = pd.read_csv('/content/Superstore.csv', encoding='latin1', engine='python')
```

```
# Checking the first few rows to understand the data structure..
df.head(8)
```

	Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	Country	City	...	Postal Code	Region	Product ID	Cate
0	1	CA-2013-152156	09-11-2013	12-11-2013	Second Class	CG-12520	Claire Gute	Consumer	United States	Henderson	...	42420	South	FUR-BO-10001798	Furr
1	2	CA-2013-152156	09-11-2013	12-11-2013	Second Class	CG-12520	Claire Gute	Consumer	United States	Henderson	...	42420	South	FUR-CH-10000454	Furr
2	3	CA-2013-138688	13-06-2013	17-06-2013	Second Class	DV-13045	Darrin Van Huff	Corporate	United States	Los Angeles	...	90036	West	OFF-LA-10000240	C Sup
3	4	US-2012-108966	11-10-2012	18-10-2012	Standard Class	SO-20335	Sean O'Donnell	Consumer	United States	Fort Lauderdale	...	33311	South	FUR-TA-10000577	Furr
4	5	US-2012-108966	11-10-2012	18-10-2012	Standard Class	SO-20335	Sean O'Donnell	Consumer	United States	Fort Lauderdale	...	33311	South	OFF-ST-10000760	C Sup
5	6	CA-2011-115812	09-06-2011	14-06-2011	Standard Class	BH-11710	Brosina Hoffman	Consumer	United States	Los Angeles	...	90032	West	FUR-FU-10001487	Furr
6	7	CA-2011-115812	09-06-2011	14-06-2011	Standard Class	BH-11710	Brosina Hoffman	Consumer	United States	Los Angeles	...	90032	West	OFF-AR-10002833	C Sup
7	8	CA-2011-115812	09-06-2011	14-06-2011	Standard Class	BH-11710	Brosina Hoffman	Consumer	United States	Los Angeles	...	90032	West	TEC-PH-10002275	Techno

8 rows × 21 columns

```
# Checking the total number of rows and columns in the dataset..
```

```
df.shape
```

```
(9994, 21)
```

```
# Checking the last few rows to inspect end-of-file consistency.
```

```
df.tail()
```

	Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	Country	City	...	Postal Code	Region	Product
9989	9990	CA-2011-110422	22-01-2011	24-01-2011	Second Class	TB-21400	Tom Boeckenhauer	Consumer	United States	Miami	...	33180	South	FUR-F100018
9990	9991	CA-2014-121258	27-02-2014	04-03-2014	Standard Class	DB-13060	Dave Brooks	Consumer	United States	Costa Mesa	...	92627	West	FUR-F100007
9991	9992	CA-2014-121258	27-02-2014	04-03-2014	Standard Class	DB-13060	Dave Brooks	Consumer	United States	Costa Mesa	...	92627	West	TEC-P100036
9992	9993	CA-2014-121258	27-02-2014	04-03-2014	Standard Class	DB-13060	Dave Brooks	Consumer	United States	Costa Mesa	...	92627	West	OFF-P100040
9993	9994	CA-2014-119914	05-05-2014	10-05-2014	Second Class	CC-12220	Chris Cortes	Consumer	United States	Westminster	...	92683	West	OFF-A100026

5 rows × 21 columns

Inspecting column data types, non-null counts, and memory usage.

df.info()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9994 entries, 0 to 9993
Data columns (total 21 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Row ID                9994 non-null  int64
1   Order ID              9994 non-null  object
2   Order Date            9994 non-null  object
3   Ship Date             9994 non-null  object
4   Ship Mode             9994 non-null  object
5   Customer ID           9994 non-null  object
6   Customer Name         9994 non-null  object
7   Segment               9994 non-null  object
8   Country               9994 non-null  object
9   City                  9994 non-null  object
10  State                 9994 non-null  object
11  Postal Code           9994 non-null  int64
12  Region                9994 non-null  object
13  Product ID            9994 non-null  object
14  Category              9994 non-null  object
15  Sub-Category          9994 non-null  object
16  Product Name          9994 non-null  object
17  Sales                  9994 non-null  float64
18  Quantity              9994 non-null  int64
19  Discount               9994 non-null  float64
20  Profit                9994 non-null  float64
dtypes: float64(3), int64(3), object(15)
memory usage: 1.6+ MB
```

Checking overall dataset summary using standard describe

df.describe()

	Row ID	Postal Code	Sales	Quantity	Discount	Profit
count	9994.000000	9994.000000	9994.000000	9994.000000	9994.000000	9994.000000
mean	4997.500000	55190.379428	229.858001	3.789574	0.156203	28.656896
std	2885.163629	32063.693350	623.245101	2.225110	0.206452	234.260108
min	1.000000	1040.000000	0.444000	1.000000	0.000000	-6599.978000
25%	2499.250000	23223.000000	17.280000	2.000000	0.000000	1.728750
50%	4997.500000	56430.500000	54.490000	3.000000	0.200000	8.666500
75%	7495.750000	90008.000000	209.940000	5.000000	0.200000	29.364000
max	9994.000000	99301.000000	22638.480000	14.000000	0.800000	8399.976000

Describing float data type columns only to inspect continuous numerical fields

df.describe(include='float64')

	Sales	Discount	Profit
count	9994.000000	9994.000000	9994.000000
mean	229.858001	0.156203	28.656896
std	623.245101	0.206452	234.260108
min	0.444000	0.000000	-6599.978000
25%	17.280000	0.000000	1.728750
50%	54.490000	0.200000	8.666500
75%	209.940000	0.200000	29.364000
max	22638.480000	0.800000	8399.976000

Describing integer data type columns only to inspect discrete numerical fields
df.describe(include='int64')

	Row ID	Postal Code	Quantity
count	9994.000000	9994.000000	9994.000000
mean	4997.500000	55190.379428	3.789574
std	2885.163629	32063.693350	2.225110
min	1.000000	1040.000000	1.000000
25%	2499.250000	23223.000000	2.000000
50%	4997.500000	56430.500000	3.000000
75%	7495.750000	90008.000000	5.000000
max	9994.000000	99301.000000	14.000000

Describing object data type columns only to inspect categorical text fields.
df.describe(include='object')

	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	Country	City	State	Region	Product ID	Category	Cate
count	9994	9994	9994	9994	9994	9994	9994	9994	9994	9994	9994	9994	9994	
unique	5009	1238	1334	4	793	793	3	1	531	49	4	1862	3	
top	CA-2014-100111	06-09-2013	16-12-2012	Standard Class	WB-21850	William Brown	Consumer	United States	New York City	California	West	OFF-PA-10001970	Office Supplies	Bi
freq	14	38	35	5968	37	37	5191	9994	915	2001	3203	19	6026	

Auditing missing values count per column.
df.isnull().sum()

	0
Row ID	0
Order ID	0
Order Date	0
Ship Date	0
Ship Mode	0
Customer ID	0
Customer Name	0
Segment	0
Country	0
City	0
State	0
Postal Code	0
Region	0
Product ID	0
Category	0
Sub-Category	0
Product Name	0
Sales	0
Quantity	0
Discount	0
Profit	0

dtype: int64

```
# Checking exact duplicate rows across the entire dataset
df.duplicated().sum()
```

```
np.int64(0)
```

```
# Converting date columns from object to datetime format
df['Order Date'] = pd.to_datetime(df['Order Date'], format='%d-%m-%Y')
df['Ship Date'] = pd.to_datetime(df['Ship Date'], format='%d-%m-%Y')
print(df[['Order Date', 'Ship Date']].dtypes)
```

```
Order Date    datetime64[ns]
Ship Date     datetime64[ns]
dtype: object
```

```
# Identifying logical anomalies where shipping date occurs before order date
invalid_ship_dates = df[df['Ship Date'] < df['Order Date']]
print("Invalid Shipping Date Count:", len(invalid_ship_dates))
```

```
Invalid Shipping Date Count: 0
```

```
# Iterating across all columns to calculate total unique counts of each column.
unique_counts = pd.DataFrame({
    'Total Unique Count': df.nunique()
})
```

```
# Displaying unique counts for all columns
unique_counts
```

Total Unique Count	
Row ID	9994
Order ID	5009
Order Date	1238
Ship Date	1334
Ship Mode	4
Customer ID	793
Customer Name	793
Segment	3
Country	1
City	531
State	49
Postal Code	631
Region	4
Product ID	1862
Category	3
Sub-Category	17
Product Name	1841
Sales	5825
Quantity	14
Discount	12
Profit	7287

```
category_sales = df.groupby("Category")["Sales"].sum()
print(category_sales)
```

```
Category
Furniture      741999.7953
Office Supplies  719047.0320
Technology     836154.0330
Name: Sales, dtype: float64
```

```
category_profit = df.groupby("Category")["Profit"].sum()
print(category_profit)
```

```
Category
Furniture      18451.2728
Office Supplies 122490.8008
Technology     145454.9481
Name: Profit, dtype: float64
```

```
# Calculate total discount per category
category_total_discount = df.groupby("Category")["Discount"].sum()

# Calculate the overall total discount
overall_total_discount = df["Discount"].sum()

# Calculate the percentage of total discount for each category
category_discount_percentage = (category_total_discount / overall_total_discount) * 100

print(category_discount_percentage)
```

```
Category
Furniture      23.630284
Office Supplies 60.713988
Technology     15.655728
Name: Discount, dtype: float64
```

```
# Analyzing Sub-Category performance to spot loss-making products
sub_category_summary = df.groupby(['Category', 'Sub-Category']).agg(
    Total_Sales=('Sales', 'sum'),
    Total_Profit=('Profit', 'sum'),
    Avg_Discount=('Discount', 'mean')
).sort_values(by='Total_Profit', ascending=False).reset_index()
```

```
# Displaying top loss-making or high-performing sub-categories
sub_category_summary
```

	Category	Sub-Category	Total_Sales	Total_Profit	Avg_Discount
0	Technology	Copiers	149528.0300	55617.8249	0.161765
1	Technology	Phones	330007.0540	44515.7306	0.154556
2	Technology	Accessories	167380.3180	41936.6357	0.078452
3	Office Supplies	Paper	78479.2060	34053.5693	0.074891
4	Office Supplies	Binders	203412.7330	30221.7633	0.372292
5	Furniture	Chairs	328449.1030	26590.1663	0.170178
6	Office Supplies	Storage	223843.6080	21278.8264	0.074704
7	Office Supplies	Appliances	107532.1610	18138.0054	0.166524
8	Furniture	Furnishings	91705.1640	13059.1436	0.138349
9	Office Supplies	Envelopes	16476.4020	6964.1767	0.080315
10	Office Supplies	Art	27118.7920	6527.7870	0.074874
11	Office Supplies	Labels	12486.3120	5546.2540	0.068681
12	Technology	Machines	189238.6310	3384.7569	0.306087
13	Office Supplies	Fasteners	3024.2800	949.5182	0.082028
14	Office Supplies	Supplies	46673.5380	-1189.0995	0.076842
15	Furniture	Bookcases	114879.9963	-3472.5560	0.211140
16	Furniture	Tables	206965.5320	-17725.4811	0.261285

```
# Basic Regional Aggregation: Summarizing Sales, Profit, and Discount by Region
region_summary = df.groupby('Region').agg(
    Total_Sales=('Sales', 'sum'),
    Total_Profit=('Profit', 'sum'),
    Avg_Discount=('Discount', 'mean')
).reset_index()

region_summary
```

	Region	Total_Sales	Total_Profit	Avg_Discount
0	Central	501239.8908	39706.3625	0.240353
1	East	678781.2400	91522.7800	0.145365
2	South	391721.9050	46749.4303	0.147253
3	West	725457.8245	108418.4489	0.109335

```
# Basic Customer Segment Aggregation: Summarizing Sales, Profit, and Discount by Segment
segment_summary = df.groupby('Segment').agg(
    Total_Sales=('Sales', 'sum'),
    Total_Profit=('Profit', 'sum'),
    Avg_Discount=('Discount', 'mean')
).reset_index()

segment_summary
```

	Segment	Total_Sales	Total_Profit	Avg_Discount
0	Consumer	1.161401e+06	134119.2092	0.158141
1	Corporate	7.061464e+05	91979.1340	0.158228
2	Home Office	4.296531e+05	60298.6785	0.147128

▼

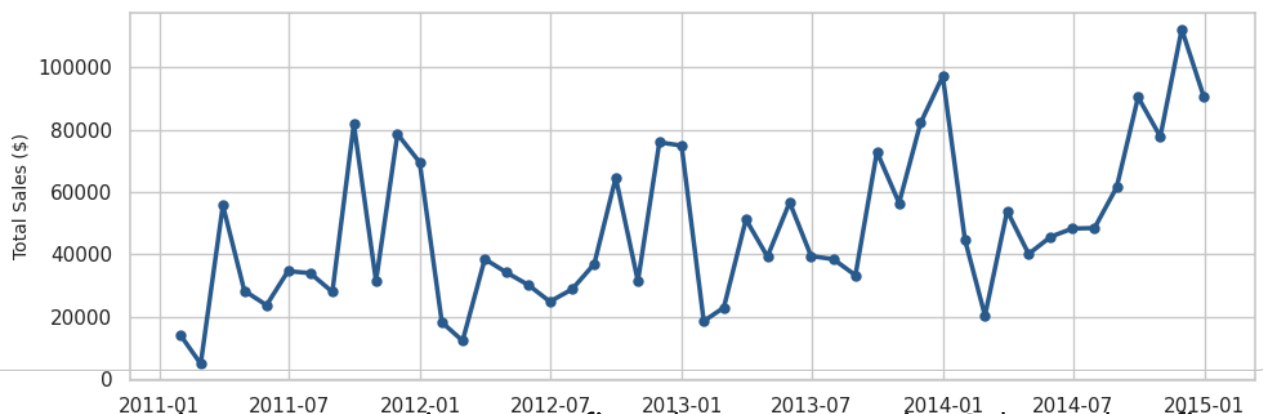
Part 1: Macro Business EDA & Revenue Diagnostics:

Objective: Analyzing baseline revenue growth trends, product category profitability, regional sales distribution, and discount threshold impacts across transaction history.

> MacroGrowth #SeasonalityCheck #TimelineBaseline:

Show code

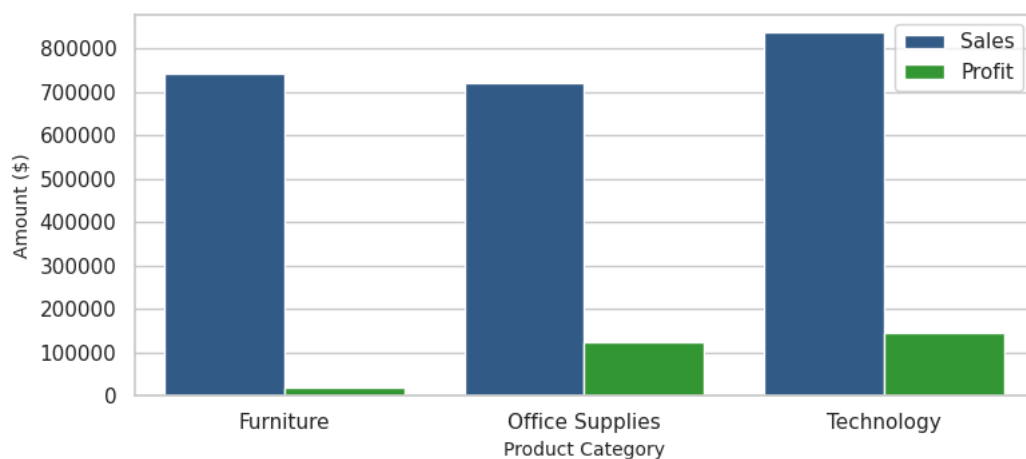
1. Monthly Revenue Growth Trend



> CategoryMargins #RevenueDrivers #ProfitLeakage:Aggregating Sales and Profit by Category

Show code

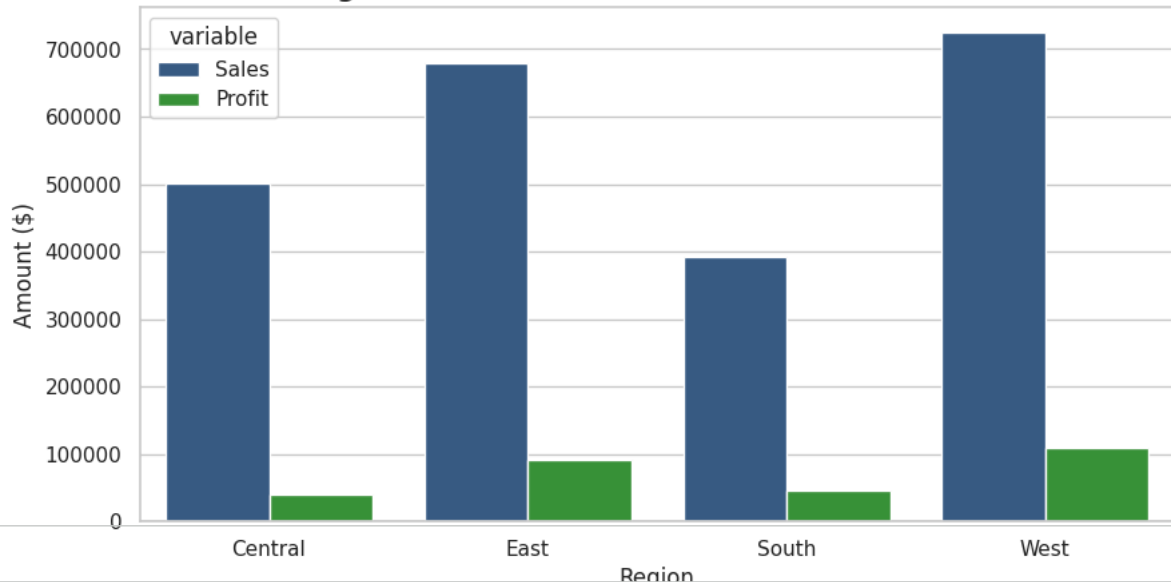
2. Category Performance: Sales vs Profit



> RegionalDistribution #TerritoryPerformance #GeographicBreakdown Bar plot for Sales and Profit distribution across Regions

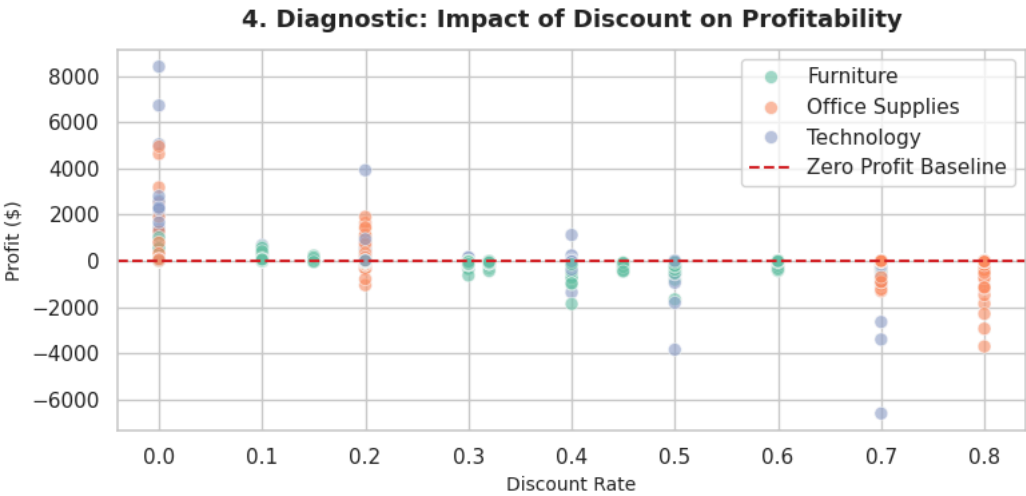
Show code

Regional Performance: Total Sales vs Total Profit



> DiscountThreshold #LossDiagnostic #ZeroProfitBaseline # Scatter plot for Discount vs Profit

Show code

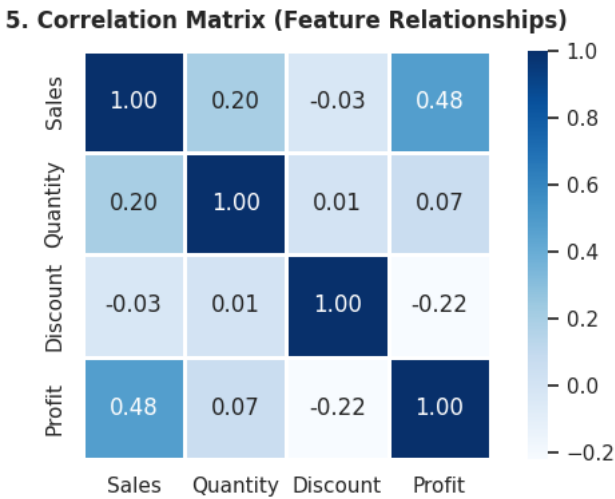


Part 2: Diagnostic EDA & Data Quality Checks

Objective: We use **Kernel Density Estimation (KDE)** alongside histograms to evaluate feature distributions, check inter-purchase intervals, and detect skewness . This validates data quality and proves the need for log-transformation before RFM scoring and K-Means Clustering.

> CorrelationMatrix #MulticollinearityCheck #FeatureInteractions

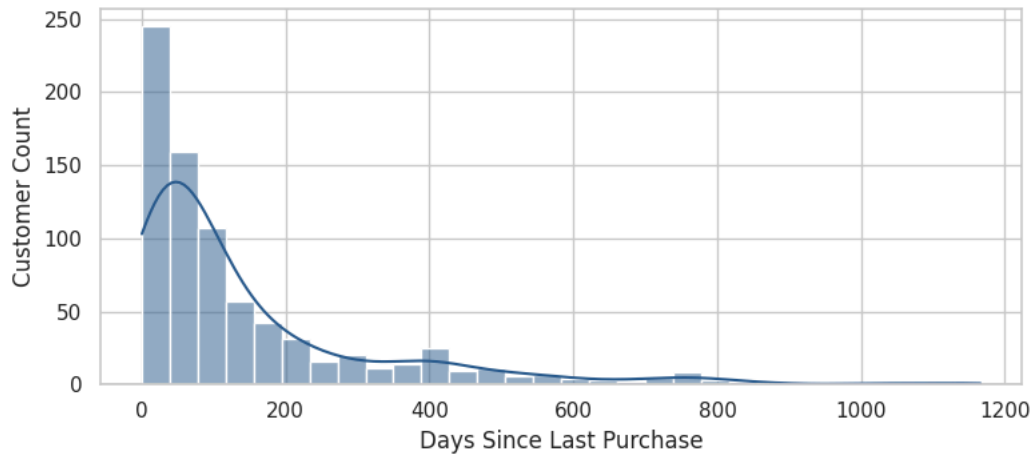
Show code



> RecencyKDE #InactivitySpread #PercentileScoringPrep

Show code

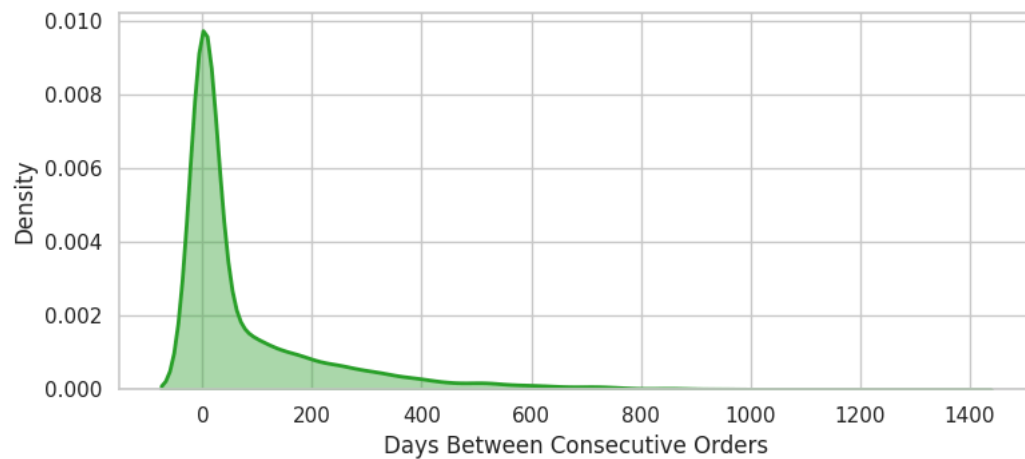
6. Recency Baseline Distribution



InterPurchaseIntervals #RepeatBehaviorKDE

[Show code](#)

7. Customer Inter-Purchase Interval Cadence



```
# One-liner to calculate skewness for all continuous features
df[['Sales', 'Quantity', 'Profit']].skew()
```

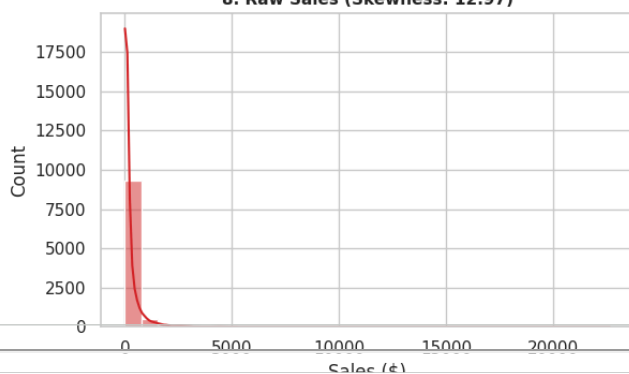
	0
Sales	12.972752
Quantity	1.278545
Profit	7.561432

dtype: float64

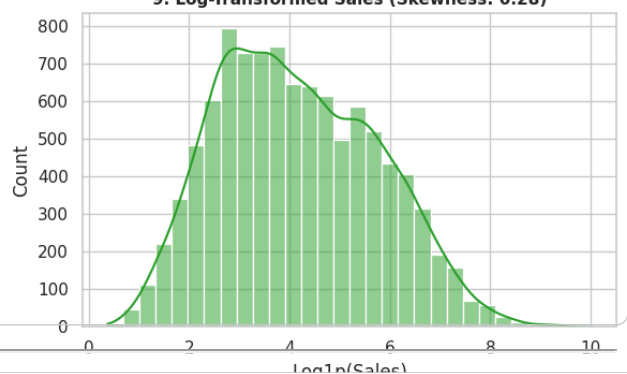
SkewnessReduction #Log1pTransformation

[Show code](#)

8. Raw Sales (Skewness: 12.97)



9. Log-Transformed Sales (Skewness: 0.28)



Part 3: Customer RFM-PQD Feature Analysis

Objective: Aggregating customer purchase patterns by Recency, Frequency, Sales (Monetary), Profit, Quantity, and Discounts to score overall buying behavior.

```
import pandas as pd
import numpy as np

# 1. Reference date set up
snapshot_date = df['Order Date'].max() + pd.Timedelta(days=1)

# 2. Aggregate extended RFM + Profit, Discount & Quantity metrics
rfm_extended = df.groupby('Customer ID').agg({
    'Order Date': lambda x: (snapshot_date - x.max()).days, # Recency
    'Order ID': 'nunique', # Frequency
    'Sales': 'sum', # Monetary (Sales)
    'Quantity': 'sum', # Total Units Purchased
    'Profit': 'sum', # Net Profit
    'Discount': 'mean' # Average Discount Rate
}).reset_index()

# Rename columns
rfm_extended.columns = [
    'Customer_ID', 'Recency', 'Frequency',
    'Monetary_Sales', 'Total_Quantity', 'Net_Profit', 'Avg_Discount'
]

# 3. Quantile Scoring (5-Tier Percentile Rank, 1-5 Scale)
# Recency (Reversed: Lower days = Higher rank)
rfm_extended['R_Score'] = pd.qcut(rfm_extended['Recency'], q=5, labels=[5, 4, 3, 2, 1])

# Frequency (Rank method handles repeated low values)
rfm_extended['F_Score'] = pd.qcut(rfm_extended['Frequency'].rank(method='first'), q=5, labels=[1, 2, 3, 4, 5])

# Monetary (Higher sales = Higher rank)
rfm_extended['M_Score'] = pd.qcut(rfm_extended['Monetary_Sales'], q=5, labels=[1, 2, 3, 4, 5])

# Quantity (Higher order volume = Higher rank)
rfm_extended['Q_Score'] = pd.qcut(rfm_extended['Total_Quantity'].rank(method='first'), q=5, labels=[1, 2, 3, 4, 5])

# 4. Composite RFM Segment & Score
rfm_extended['RFM_Segment'] = (
    rfm_extended['R_Score'].astype(str) +
    rfm_extended['F_Score'].astype(str) +
    rfm_extended['M_Score'].astype(str)
)

rfm_extended['Composite_RFM_Score'] = (
    rfm_extended['R_Score'].astype(int) +
    rfm_extended['F_Score'].astype(int) +
    rfm_extended['M_Score'].astype(int)
)

rfm_extended.head(10)
```

	Customer_ID	Recency	Frequency	Monetary_Sales	Total_Quantity	Net_Profit	Avg_Discount	R_Score	F_Score	M_Score	Q_Score
0	AA-10315	185	5	5563.560	30	-362.8825	0.090909	2	2	5	
1	AA-10375	20	9	1056.390	41	277.3824	0.080000	5	5	2	
2	AA-10480	260	4	1790.512	36	435.8274	0.016667	1	1	3	
3	AA-10645	56	6	5086.935	64	857.8033	0.063889	3	3	5	
4	AB-10015	416	3	886.156	13	129.3465	0.066667	1	1	1	
5	AB-10060	55	8	7755.620	56	2054.5885	0.044444	3	4	5	
6	AB-10105	42	10	14473.571	73	5444.8055	0.240000	4	5	5	
7	AB-10150	42	5	966.710	37	313.6597	0.133333	4	2	2	
8	AB-10165	26	8	1113.838	48	220.8130	0.135714	5	4	2	
9	AB-10255	167	9	914.532	39	264.5675	0.100000	2	5	1	

```
# Custom function to map RFM Scores into Business Segments
def segment_customer(df):
```

```

r = int(df['R_Score'])
f = int(df['F_Score'])
m = int(df['M_Score'])

if r >= 4 and f >= 4 and m >= 4:
    return 'Champions'
elif r >= 3 and f >= 3 and m >= 3:
    return 'Loyal Customers'
elif r >= 4 and f <= 2:
    return 'Recent Buyers / Promising'
elif r <= 2 and f >= 4:
    return 'At Risk (High Vol / Churn Risk)'
elif r <= 2 and f <= 2 and m <= 2:
    return 'Lost Customers'
else:
    return 'Needs Attention'

# Apply Segment Mapping
rfm_extended['Customer_Persona'] = rfm_extended.apply(segment_customer, axis=1)

# Segment Count Summary
rfm_extended['Customer_Persona'].value_counts().reset_index()

```

	Customer_Persona	count
0	Needs Attention	242
1	Loyal Customers	159
2	Lost Customers	117
3	Champions	106
4	Recent Buyers / Promising	89
5	At Risk (High Vol / Churn Risk)	80

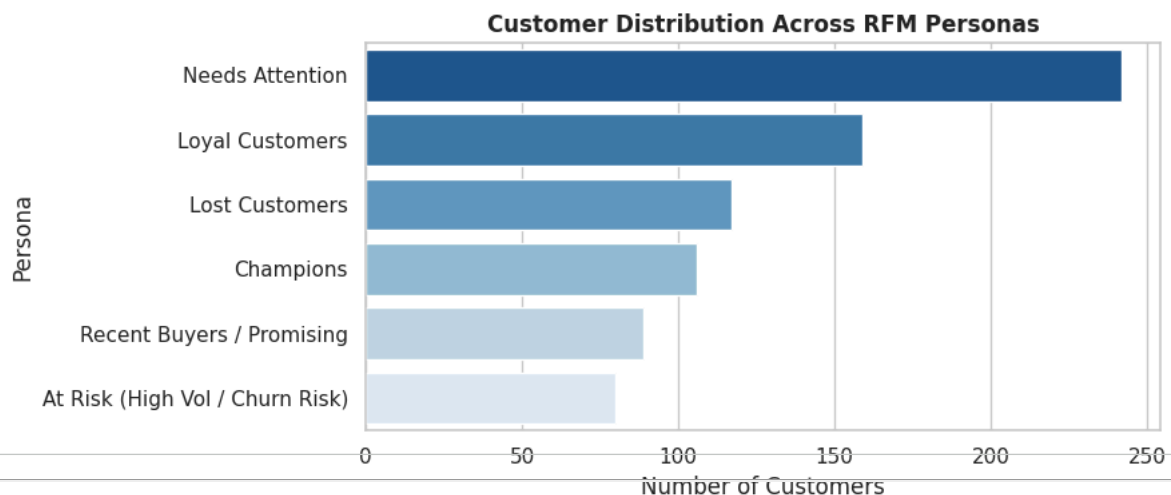
Summary Chart: Mapped Customer Segments

```

import matplotlib.pyplot as plt
import seaborn as sns

# Clean 3-Line Persona Bar Plot
plt.figure(figsize=(9, 4), dpi=100)
sns.countplot(data=rfm_extended, y='Customer_Persona', order=rfm_extended['Customer_Persona'].value_counts().index, palette='magma')
plt.title("Customer Distribution Across RFM Personas", fontweight='bold')
plt.xlabel("Number of Customers")
plt.ylabel("Persona")
plt.tight_layout()
plt.show()

```



Part 4: Pre-PCA Skewness Mitigation & Standardization

Technical Objective: Log-transforming continuous metrics (`np.log1p`) to eliminate right-skewness and achieve normal distribution symmetry, followed by `StandardScaler` prior to PCA execution.

```
# Skewness Mitigation: Yeo-Johnson Power Transformation (Handles Negatives & Extreme Skewness)
from sklearn.preprocessing import PowerTransformer

#selecting the continous features.
features = ['Recency', 'Frequency', 'Monetary_Sales', 'Total_Quantity', 'Net_Profit']
X = rfm_extended[features].copy()

# 2. Yeo-Johnson Transformer (Handles zero and negative values automatically)
pt = PowerTransformer(method='yeo-johnson')
X_log_data = pt.fit_transform(X)

# Convert into the Dataframe.
X_log = pd.DataFrame(X_log_data, columns=features)

# 3. Diagnostic Skewness Comparison Table
skew_df = pd.DataFrame({
    'Before_Transform': X.skew(),
    'After_Transform': X_log.skew()
}).round(2)

print("Skewness Diagnostics Check")
print(skew_df)
```

Skewness Diagnostics Check		
	Before_Transform	After_Transform
Recency	2.28	-0.01
Frequency	0.36	-0.02
Monetary_Sales	2.48	0.03
Total_Quantity	0.80	-0.01
Net_Profit	1.83	1.90

```
# Feature Standardization using StandardScaler
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_log)

# Scaling check
print("Scaled Matrix Shape:", X_scaled.shape)
```

Scaled Matrix Shape: (793, 5)

```
# PCA Execution & Variance Loss Verification
from sklearn.decomposition import PCA

# 1. Initialize PCA to retain 90% of total variance
pca = PCA(n_components=0.90)
X_pca = pca.fit_transform(X_scaled)

# 2. Check Individual & Cumulative Variance
var_explained = pca.explained_variance_ratio_ * 100
total_var = np.sum(var_explained)

print(f"Original Features Count : {X_scaled.shape[1]}")
print(f"Retained PCA Components : {X_pca.shape[1]}")
print(f"Total Information Kept : {total_var:.2f}%")
print(f>Data Loss (Negligible) : {100 - total_var:.2f}%")
print("-" * 40)

# Component-wise breakup
for i, var in enumerate(var_explained, 1):
    print(f"PC{i} Explains : {var:.2f}% Variance")
```

Original Features Count	: 5
Retained PCA Components	: 4
Total Information Kept	: 96.73%
Data Loss (Negligible)	: 3.27%

PC1 Explains	: 52.92% Variance
PC2 Explains	: 20.77% Variance
PC3 Explains	: 15.67% Variance
PC4 Explains	: 7.37% Variance

```
# Create Reduced PCA DataFrame & Feature Loadings Matrix

# 1. Converting Reduced PCA Matrix into clean DataFrame
pc_colnames = [f'PC{i+1}' for i in range(X_pca.shape[1])]
X_pca_df = pd.DataFrame(X_pca, columns=pc_colnames, index=rfm.index)

# 2. Feature Loadings (Weights) Matrix Extract karna
loadings = pd.DataFrame(
    pca.components_.T,
    columns=pc_colnames,
```

```

        index=features
    ).round(3)

    print(" Reduced PCA Dataset Sample (First 3 Rows)")
    print(X_pca_df.head(3))

    print("\nPCA Feature Loadings Matrix (Which Feature Controls Which PC)")
    print(loadings)

```

```

Reduced PCA Dataset Sample (First 3 Rows)
      PC1      PC2      PC3      PC4
0 -0.422170  0.278752  0.734317  1.357414
1  0.354313 -1.064014 -0.561138 -0.949577
2 -1.037101  0.826975  0.364883  0.200066

```

```

PCA Feature Loadings Matrix (Which Feature Controls Which PC)
      PC1      PC2      PC3      PC4
Recency    -0.268  0.597  0.744 -0.117
Frequency   0.532 -0.214  0.230 -0.540
Monetary_Sales 0.515  0.250  0.078  0.759
Total_Quantity 0.555 -0.088  0.322 -0.083
Net_Profit    0.267  0.726 -0.532 -0.335

```

Part 5: K-Means Clustering - Optimal k Selection

Evaluating $k \in [2, 8]$ on reduced PCA features (`X_pca_df`) using **Elbow Method (WCSS)** and **Silhouette Scores** to identify optimal customer segments.

```

# Cell 1: Calculate WCSS and Silhouette Scores for k = 2 to 8

from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
import matplotlib.pyplot as plt

wcss = []
silhouette_scores = []
k_range = range(2, 9)

# Loop to evaluate KMeans for each k
for k in k_range:
    kmeans = KMeans(n_clusters=k, init='k-means++', random_state=42, n_init=10)
    kmeans.fit(X_pca_df)

    wcss.append(kmeans.inertia_)
    silhouette_scores.append(silhouette_score(X_pca_df, kmeans.labels_))

# Plotting Elbow Curve and Silhouette Scores
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4), dpi=100)

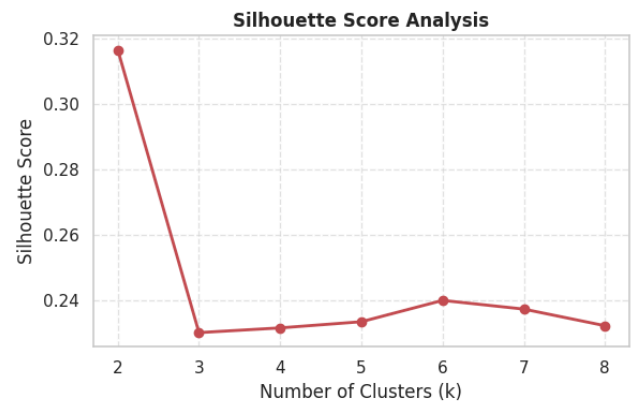
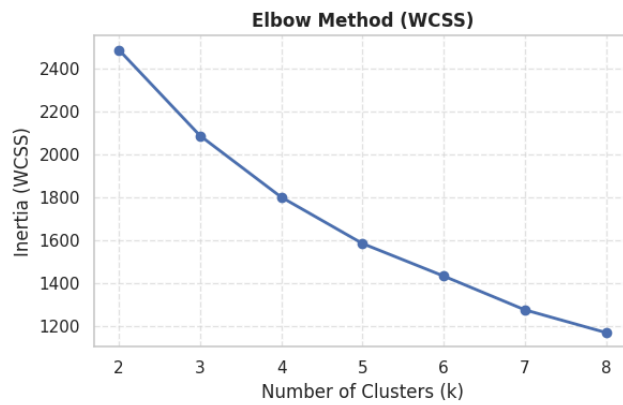
ax1.plot(k_range, wcss, 'bo-', linewidth=2, markersize=6)
ax1.set_title('Elbow Method (WCSS)', fontweight='bold')
ax1.set_xlabel('Number of Clusters (k)')
ax1.set_ylabel('Inertia (WCSS)')
ax1.grid(True, linestyle='--', alpha=0.5)

ax2.plot(k_range, silhouette_scores, 'ro-', linewidth=2, markersize=6)
ax2.set_title('Silhouette Score Analysis', fontweight='bold')
ax2.set_xlabel('Number of Clusters (k)')
ax2.set_ylabel('Silhouette Score')
ax2.grid(True, linestyle='--', alpha=0.5)

plt.tight_layout()
plt.show()

# Print Scores
for k, score in zip(k_range, silhouette_scores):
    print(f"Clusters (k={k}) -> Silhouette Score: {score:.4f}")

```



```
Clusters (k=2) -> Silhouette Score: 0.3166
Clusters (k=3) -> Silhouette Score: 0.2303
Clusters (k=4) -> Silhouette Score: 0.2317
Clusters (k=5) -> Silhouette Score: 0.2336
Clusters (k=6) -> Silhouette Score: 0.2401
```

```
# Cell 2: Train Final K-Means Model and Map Cluster Labels to PCA Data
```

```
# Set optimal k (e.g., k=4 based on Elbow & Silhouette analysis)
optimal_k = 4
```

```
kmeans_final = KMeans(n_clusters=optimal_k, init='k-means++', random_state=42, n_init=10)
cluster_labels = kmeans_final.fit_predict(X_pca_df)
```

```
# Attach Cluster Labels to both Reduced PCA DataFrame and Original RFM DataFrame
X_pca_df['Cluster'] = cluster_labels
rfm['Cluster'] = cluster_labels
```

```
# 1. Clear View of Reduced PCA Data Frame with Cluster Labels
print("Reduced PCA Data Frame with Cluster Labels (First 5 Rows)")
print(X_pca_df.head())
```

```
print("\nCluster Count Distribution")
print(X_pca_df['Cluster'].value_counts().sort_index().to_frame(name='Total Customers'))
```

```
Reduced PCA Data Frame with Cluster Labels (First 5 Rows)
```

	PC1	PC2	PC3	PC4	Cluster
0	-0.422170	0.278752	0.734317	1.357414	0
1	0.354313	-1.064014	-0.561138	-0.949577	3
2	-1.037101	0.826975	0.364883	0.200066	0
3	1.120065	0.453330	-0.182037	0.585005	3
4	-2.559678	0.880312	0.272394	0.136800	1

```
Cluster Count Distribution
Total Customers
```

Cluster	Total Customers
0	313
1	147
2	70
3	263

```
# Cell 3: Clear 2D Visual Scatter Plot on PCA Space

import seaborn as sns

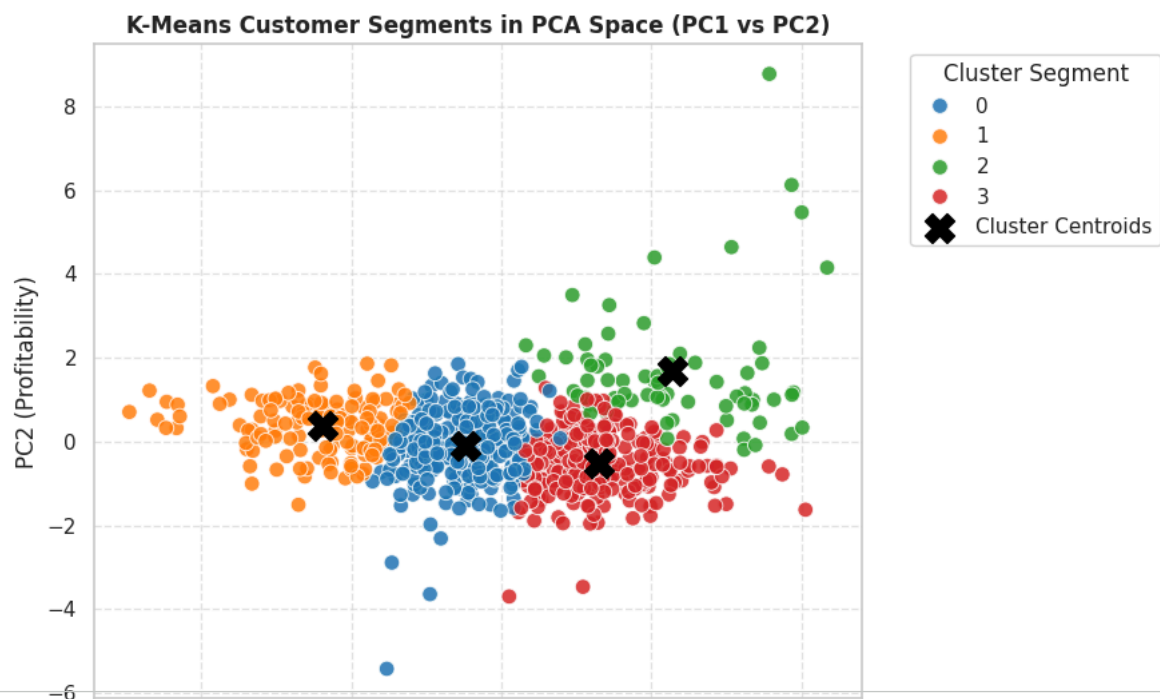
plt.figure(figsize=(9, 6), dpi=100)

# Scatter plot of PC1 vs PC2 colored by Cluster label
sns.scatterplot(
    x='PC1', y='PC2', hue='Cluster', palette='tab10',
    data=X_pca_df, s=70, alpha=0.85
)

# Plotting Centroids on PC1 & PC2 axes
centroids = kmeans_final.cluster_centers_
plt.scatter(
    centroids[:, 0], centroids[:, 1],
    s=250, c='black', marker='X', label='Cluster Centroids'
)

plt.title('K-Means Customer Segments in PCA Space (PC1 vs PC2)', fontweight='bold', fontsize=12)
plt.xlabel('PC1 (Customer Activity & Volume)')
plt.ylabel('PC2 (Profitability)')
plt.legend(title='Cluster Segment', bbox_to_anchor=(1.05, 1), loc='upper left')
plt.grid(True, linestyle='--', alpha=0.5)

plt.tight_layout()
plt.show()
```



```
# Cell 4: Calculate Mean RFM Values per Cluster

# Ensure rfm_extended also has the Cluster labels for comprehensive profiling
# Assuming order is preserved as both rfm and rfm_extended are derived from customer-level aggregations
rfm_extended['Cluster'] = rfm['Cluster']

# Select only the relevant numeric columns for profiling
profiling_cols = ['Recency', 'Frequency', 'Monetary_Sales', 'Total_Quantity', 'Net_Profit', 'Avg_Discount']
cluster_profile = rfm_extended.groupby('Cluster')[profiling_cols].mean().round(2)
print("Customer Segment Profiles (Averages)")
print(cluster_profile)
```

```
Customer Segment Profiles (Averages)
      Recency  Frequency  Monetary_Sales  Total_Quantity  Net_Profit \
Cluster
0         132.99        5.51         2065.55           38.41        162.51
1         337.10        3.21          731.56           19.18         53.80
2         144.51        8.11        8323.47           73.03       2136.45
3          60.51        8.54        3652.10           68.13        296.85

      Avg_Discount
Cluster
0             0.16
1             0.17
2             0.11
3             0.16
```

Phase 2: Customer Lifetime Value (CLV) Analysis

Objective: Quantify the financial value and lifetime profit contribution of each of the 4 K-Means RFM clusters.

Key Metrics Computed:

- **Average Order Value (AOV):** Total Revenue ÷ Total Order Frequency.
- **Annual Customer Value:** AOV × Annual Order Frequency.
- **4-Year Estimated CLV:** Total expected monetary spend over a 48-month retention window.
- **4-Year Profit Contribution:** Total net profit margin generated per customer segment.

```
# Cell 6: Customer Lifetime Value (CLV) Analysis by Segments.

# 1. Aggregate RFM and Profit Metrics by Customer Segment
# Using rfm_extended which contains the full set of metrics
clv_summary = rfm_extended.groupby('Cluster').agg(
    Customer_Count=('Monetary_Sales', 'count'),
    Avg_Recency=('Recency', 'mean'),
    Avg_Frequency=('Frequency', 'mean'),
    Avg_Monetary=('Monetary_Sales', 'mean'),
    Avg_Net_Profit=('Net_Profit', 'mean')
).reset_index()

# 2. Compute Average Order Value (AOV)
clv_summary['AOV'] = clv_summary['Avg_Monetary'] / clv_summary['Avg_Frequency']

# 3. Compute Customer Value (CV) per Year
clv_summary['Annual_Customer_Value'] = clv_summary['AOV'] * clv_summary['Avg_Frequency']

# 4. Estimate 2-Year Historical CLV & Expected Profit Contribution
clv_summary['Est_4Yr_CLV'] = clv_summary['Annual_Customer_Value'] * 4
clv_summary['Est_4Yr_Profit'] = clv_summary['Avg_Net_Profit'] * 4

# 5. Format Columns for Clear Presentation
clv_summary['AOV_fmt'] = clv_summary['AOV'].map('${:,.2f}'.format)
clv_summary['Avg_Monetary_fmt'] = clv_summary['Avg_Monetary'].map('${:,.2f}'.format)
clv_summary['Est_4Yr_CLV_fmt'] = clv_summary['Est_4Yr_CLV'].map('${:,.2f}'.format)
clv_summary['Est_4Yr_Profit_fmt'] = clv_summary['Est_4Yr_Profit'].map('${:,.2f}'.format)

# Display Final Table (Using Cluster as the identifier)
print("SEGMENT-WISE CUSTOMER LIFETIME VALUE (CLV) SUMMARY")
clv_summary[['Cluster', 'Customer_Count', 'AOV_fmt', 'Avg_Monetary_fmt', 'Est_4Yr_CLV_fmt', 'Est_4Yr_Profit_fmt']]
```

SEGMENT-WISE CUSTOMER LIFETIME VALUE (CLV) SUMMARY						
Cluster	Customer_Count	AOV_fmt	Avg_Monetary_fmt	Est_4Yr_CLV_fmt	Est_4Yr_Profit_fmt	
0	0	313	\$375.01	\$2,065.55	\$8,262.20	\$650.06
1	1	147	\$227.84	\$731.56	\$2,926.23	\$215.18
2	2	70	\$1,025.78	\$8,323.47	\$33,293.86	\$8,545.79
3	3	263	\$427.84	\$3,652.10	\$14,608.40	\$1,187.39

```
# Cell 7: Plot CLV and Profit Comparisons

# 1. Prepare data for plotting by ensuring numeric types and segment names
clv_summary_plot = clv_summary.copy()
# Use 'Cluster' as the name since 'Segment_Name' isn't defined
clv_summary_plot['Segment_Name'] = clv_summary_plot['Cluster'].apply(lambda x: f'Cluster {x}')

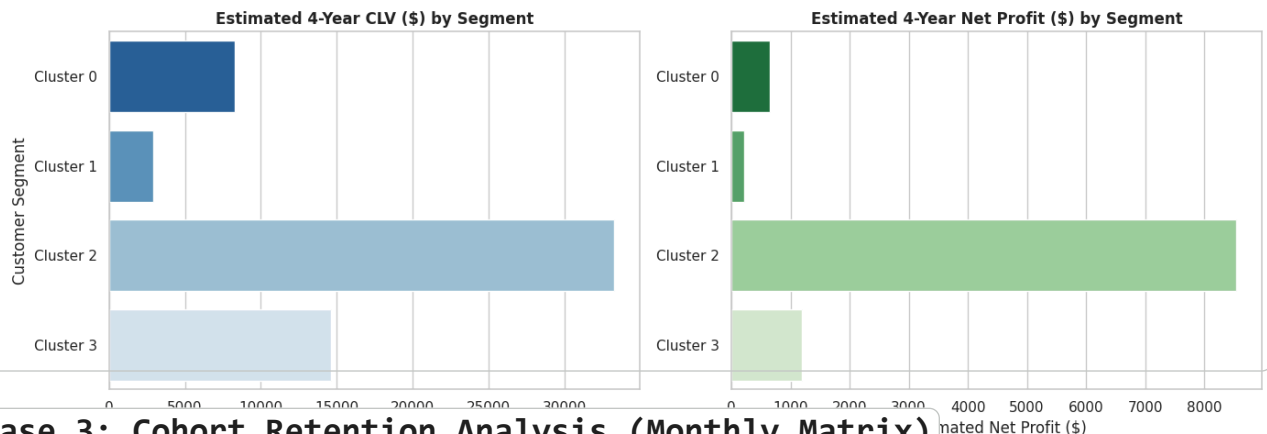
# 2. Create subplots for CLV and Net Profit
fig, axes = plt.subplots(1, 2, figsize=(14, 5), dpi=100)

# Graph 1: 4-Year Estimated CLV by Segment
sns.barplot(
    x='Est_4Yr_CLV',
    y='Segment_Name',
    data=clv_summary_plot,
    ax=axes[0],
    palette='Blues_r',
    hue='Segment_Name',
    legend=False
)
axes[0].set_title('Estimated 4-Year CLV ($) by Segment', fontweight='bold', fontsize=12)
axes[0].set_xlabel('Estimated CLV ($)')
axes[0].set_ylabel('Customer Segment')

# Graph 2: 4-Year Net Profit Contribution by Segment
```

```
sns.barplot(
    x='Est_4Yr_Profit',
    y='Segment_Name',
    data=clv_summary_plot,
    ax=axes[1],
    palette='Greens_r',
    hue='Segment_Name',
    legend=False
)
axes[1].set_title('Estimated 4-Year Net Profit ($) by Segment', fontweight='bold', fontsize=12)
axes[1].set_xlabel('Estimated Net Profit ($)')
axes[1].set_ylabel('')

plt.tight_layout()
plt.show()
```



Phase 3: Cohort Retention Analysis (Monthly Matrix)

Objective: Track customer retention rates over time by grouping customers into acquisition cohorts based on their first purchase month.

Key Concepts:

- **Cohort Group:** Month of first purchase.
- **Cohort Index:** Number of months elapsed since the first purchase (0, 1, 2, ..., n).
- **Retention Rate:** Percentage of the original cohort returning in subsequent months.

Cell 8: Cohort Retention Analysis and Heatmap Plot

1. Ensure OrderDate is in datetime format (Using your original order dataset)
`df['OrderDate'] = pd.to_datetime(df['Order Date'])`

2. Extract Year-Month for transaction date
`df['OrderMonth'] = df['OrderDate'].dt.to_period('M')`

3. Determine First Purchase Month for each customer (Cohort Group)
`df['CohortGroup'] = df.groupby('Customer ID')['OrderDate'].transform('min').dt.to_period('M')`

4. Calculate Cohort Index (Months elapsed since first purchase)
`df['CohortIndex'] = (df['OrderMonth'].dt.year - df['CohortGroup'].dt.year) * 12 + \`
`(df['OrderMonth'].dt.month - df['CohortGroup'].dt.month)`

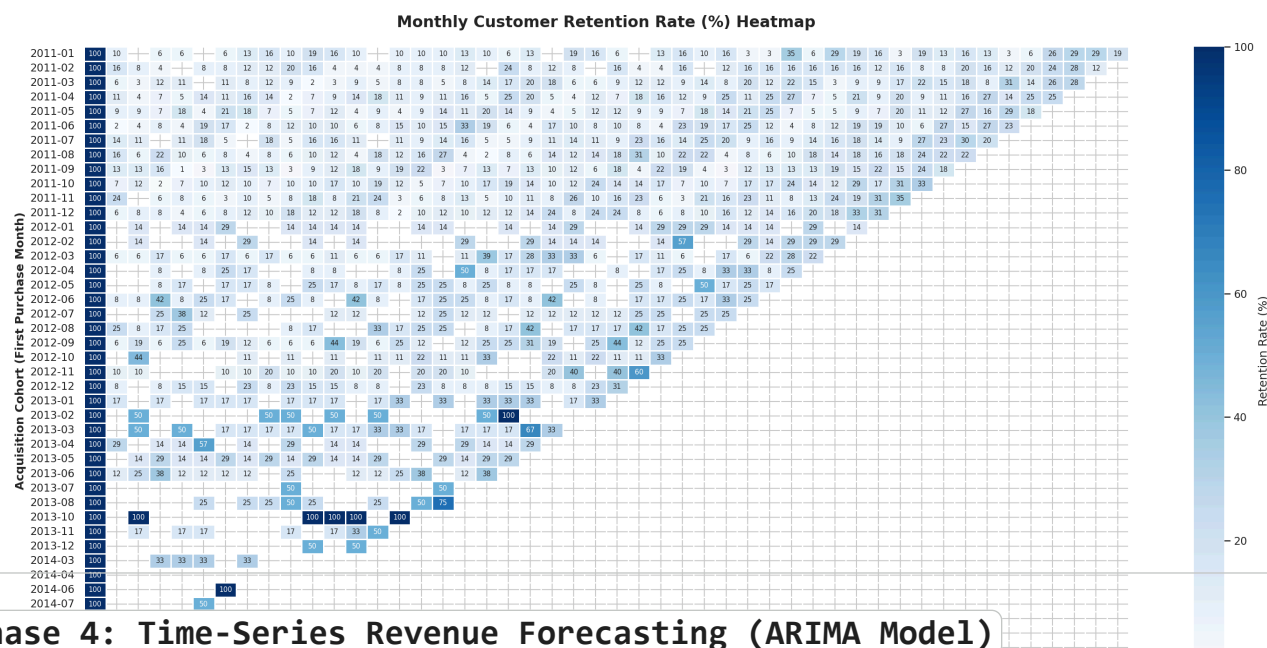
5. Build Cohort Matrix (Count of Unique Customers)
`cohort_data = df.groupby(['CohortGroup', 'CohortIndex'])['Customer ID'].nunique().reset_index()`
`cohort_matrix = cohort_data.pivot(index='CohortGroup', columns='CohortIndex', values='Customer ID')`

6. Calculate Retention Rate Percentage (%)
`cohort_size = cohort_matrix.iloc[:, 0]`
`retention_matrix = cohort_matrix.divide(cohort_size, axis=0) * 100`

7. Plot Cohort Retention Heatmap
 # Updated Cell 8 Plotting Code (Clean & Readable Version)
`plt.figure(figsize=(20, 10), dpi=150)` # Sizing increase ki hai
`plt.title('Monthly Customer Retention Rate (%) Heatmap', fontsize=16, fontweight='bold', pad=20)`

```
sns.heatmap(
    retention_matrix,
    annot=True,
    fmt='.0f',          # Decimal points hata diye (.0f) taaki clean integers dikhein
    annot_kws={'size': 7}, # Numbers ka font size chhota kar diya
    cmap='Blues',
    cbar_kws={'label': 'Retention Rate (%)'},
    linewidths=0.5      # Cells ke beech clean grid border ke liye
)
```

```
plt.xlabel('Months Elapsed (Cohort Index)', fontweight='bold', fontsize=12)
plt.ylabel('Acquisition Cohort (First Purchase Month)', fontweight='bold', fontsize=12)
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()
```



Phase 4: Time-Series Revenue Forecasting (ARIMA Model)

Objective: Forecast upcoming monthly sales trends to help inventory management align with predicted customer demand across clusters.

Key Steps:

1. Resample daily sales data to Monthly Total Sales (MS).
2. Train ARIMA (AutoRegressive Integrated Moving Average) model.
3. Forecast revenue for the next 6 to 12 months with confidence intervals.

```
# Cell 9A: Basic Time-Series Exploration & Seasonal Decomposition
import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.tsa.seasonal import seasonal_decompose

# 1. Monthly Aggregate Sales
ts_data = df.set_index('OrderDate')['Sales'].resample('MS').sum()

# 2. Seasonal Decomposition (Trend + Seasonality + Noise)
decomposition = seasonal_decompose(ts_data, model='additive')

# 3. Plotting Baseline Components
fig, axes = plt.subplots(4, 1, figsize=(12, 8), sharex=True, dpi=100)

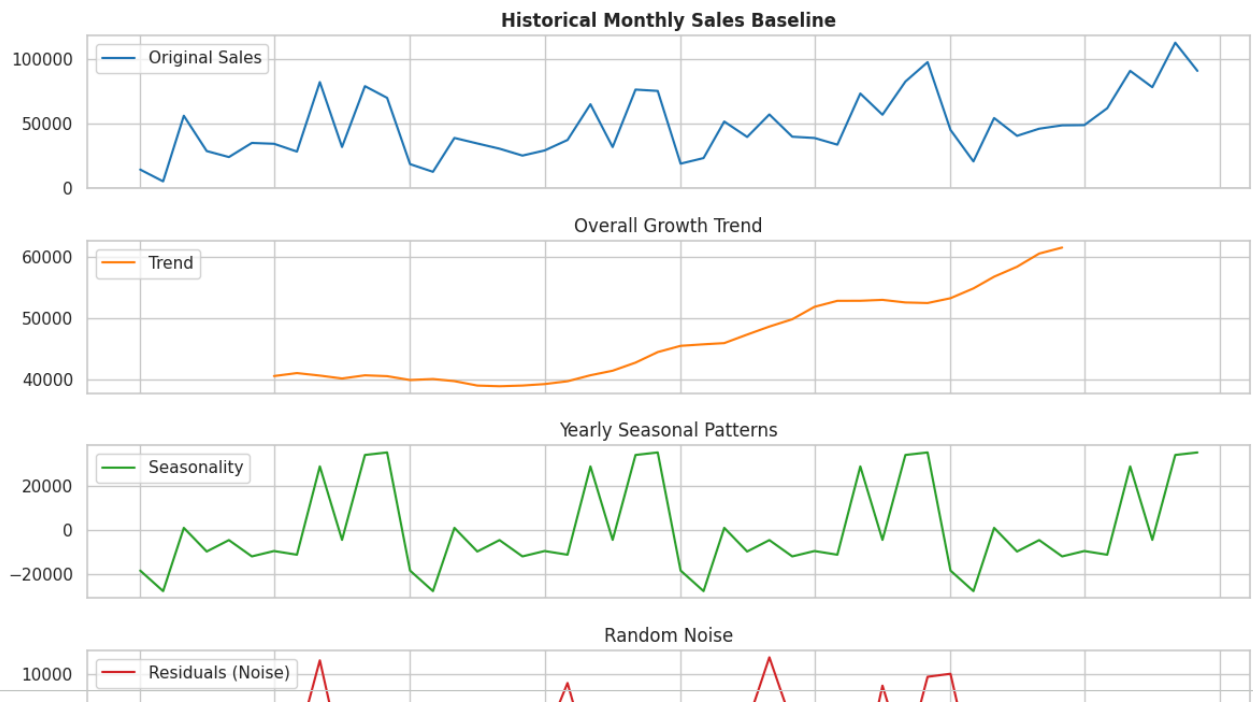
axes[0].plot(ts_data, label='Original Sales', color='#1f77b4')
axes[0].set_title('Historical Monthly Sales Baseline', fontweight='bold')
axes[0].legend(loc='upper left')

axes[1].plot(decomposition.trend, label='Trend', color='#ff7f0e')
axes[1].set_title('Overall Growth Trend')
axes[1].legend(loc='upper left')

axes[2].plot(decomposition.seasonal, label='Seasonality', color='#2ca02c')
axes[2].set_title('Yearly Seasonal Patterns')
axes[2].legend(loc='upper left')

axes[3].plot(decomposition.resid, label='Residuals (Noise)', color='#d62728')
axes[3].set_title('Random Noise')
axes[3].legend(loc='upper left')

plt.tight_layout()
plt.show()
```



```
!pip install pmdarima
import pandas as pd
import matplotlib.pyplot as plt
from pmdarima import auto_arima
from sklearn.metrics import mean_squared_error
import numpy as np

# Step 1: Create a monthly aggregate sales time series (similar to cell XXz403fz3MX6)
ts_data = df.set_index('Order Date')['Sales'].resample('MS').sum()
col_name = 'Sales'

# Step 2: Split the data into Train (2011-2013) and Test (2014) using ts_data
train_data = ts_data['2011':'2013']
test_data = ts_data['2014']

print("Train data shape:", train_data.shape)
print("Test data shape:", test_data.shape)

# Step 3: Train the Seasonal ARIMA Model using Auto-ARIMA with seasonality (m=12)
auto_model = auto_arima(train_data,
                        start_p=1, start_q=1,
                        max_p=3, max_q=3,
                        start_d=1, max_d=2,
                        seasonal=True, # Enable seasonality to capture seasonal trends
                        m=12, # Yearly seasonal cycle for monthly data (12 months)
                        trace=True,
                        error_action='ignore',
                        suppress_warnings=True,
                        stepwise=True)

print("\nBest ARIMA Order:", auto_model.order)
print("Best Seasonal Order:", auto_model.seasonal_order)

# Step 4: Generate forecast for the test period (2014)
forecast_result = auto_model.predict(n_periods=len(test_data))
forecasted_values = pd.Series(forecast_result, index=test_data.index)

# Step 5: Calculate RMSE to check quantitative accuracy
rmse = np.sqrt(mean_squared_error(test_data, forecasted_values))
print(f"Test RMSE for 2014 Prediction (Seasonal): {rmse}")

# Step 6: Plot Train Data vs Actual 2014 Data vs Predicted 2014 Data
plt.figure(figsize=(10, 5))
plt.plot(train_data.index, train_data, label='Train Data (2011-2013)', color='blue')
plt.plot(test_data.index, test_data, label='Actual 2014 Data (Sales)', color='green')
plt.plot(test_data.index, forecasted_values, label='Predicted 2014 Data (Seasonal ARIMA)', color='red', linestyle='--')
plt.title('Seasonal ARIMA Backtesting: Train (2011-2013) vs Test (2014)')
plt.xlabel('Timeline')
plt.ylabel('Sales')
plt.legend()
plt.show()
```

```

Requirement already satisfied: pmdarima in /usr/local/lib/python3.13/dist-packages (2.1.1)
Requirement already satisfied: joblib>=0.11 in /usr/local/lib/python3.13/dist-packages (from pmdarima) (1.6.0)
Requirement already satisfied: Cython!=0.29.18,!=0.29.31,>=0.29 in /usr/local/lib/python3.13/dist-packages (from pmdarima)
Requirement already satisfied: numpy>=1.21.6 in /usr/local/lib/python3.13/dist-packages (from pmdarima) (2.1.3)
Requirement already satisfied: pandas>=0.19 in /usr/local/lib/python3.13/dist-packages (from pmdarima) (2.2.3)
Requirement already satisfied: scikit-learn>=0.22 in /usr/local/lib/python3.13/dist-packages (from pmdarima) (1.6.1)
Requirement already satisfied: scipy>=1.13.0 in /usr/local/lib/python3.13/dist-packages (from pmdarima) (1.16.3)
Requirement already satisfied: statsmodels>=0.14.5 in /usr/local/lib/python3.13/dist-packages (from pmdarima) (0.15.0)
Requirement already satisfied: urllib3 in /usr/local/lib/python3.13/dist-packages (from pmdarima) (2.5.0)
Requirement already satisfied: setuptools!=50.0.0,>=42 in /usr/local/lib/python3.13/dist-packages (from pmdarima) (80.10.2)
Requirement already satisfied: packaging>=17.1 in /usr/local/lib/python3.13/dist-packages (from pmdarima) (26.3)
Requirement already satisfied: cloudpickle>=3.0 in /usr/local/lib/python3.13/dist-packages (from joblib>=0.11->pmdarima) (
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.13/dist-packages (from pandas>=0.19->pmdarima)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.13/dist-packages (from pandas>=0.19->pmdarima) (2025
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.13/dist-packages (from pandas>=0.19->pmdarima) (20
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.13/dist-packages (from scikit-learn>=0.22->p
Requirement already satisfied: patsy>=0.5.6 in /usr/local/lib/python3.13/dist-packages (from statsmodels>=0.14.5->pmdarima)
Requirement already satisfied: formulaic>=1.1.0 in /usr/local/lib/python3.13/dist-packages (from statsmodels>=0.14.5->pmdarima)
Requirement already satisfied: interface-meta>=1.2.0 in /usr/local/lib/python3.13/dist-packages (from formulaic>=1.1.0->st
Requirement already satisfied: narwhals>=1.17 in /usr/local/lib/python3.13/dist-packages (from formulaic>=1.1.0->statsmode
Requirement already satisfied: typing-extensions>=4.2.0 in /usr/local/lib/python3.13/dist-packages (from formulaic>=1.1.0-
Requirement already satisfied: wrapt>=1.17.0rc1 in /usr/local/lib/python3.13/dist-packages (from formulaic>=1.1.0->statsmo
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.13/dist-packages (from python-dateutil>=2.8.2->pandas>=0
Train data shape: (36,)
Test data shape: (12,)
Performing stepwise search to minimize aic
ARIMA(1,0,1)(1,0,1)[12] intercept : AIC=819.840, Time=0.58 sec
ARIMA(0,0,0)(0,0,0)[12] intercept : AIC=829.382, Time=0.03 sec
ARIMA(1,0,0)(1,0,0)[12] intercept : AIC=823.174, Time=0.43 sec
ARIMA(0,0,1)(0,0,1)[12] intercept : AIC=821.718, Time=0.27 sec
ARIMA(0,0,0)(0,0,0)[12] : AIC=881.959, Time=0.03 sec
ARIMA(1,0,1)(0,0,1)[12] intercept : AIC=822.840, Time=1.27 sec
ARIMA(1,0,1)(1,0,0)[12] intercept : AIC=817.840, Time=0.87 sec
ARIMA(1,0,1)(0,0,0)[12] intercept : AIC=831.277, Time=0.23 sec
ARIMA(1,0,1)(2,0,0)[12] intercept : AIC=819.779, Time=0.47 sec
ARIMA(1,0,1)(2,0,1)[12] intercept : AIC=821.529, Time=1.65 sec
ARIMA(0,0,1)(1,0,0)[12] intercept : AIC=828.889, Time=0.95 sec
ARIMA(2,0,1)(1,0,0)[12] intercept : AIC=inf, Time=3.07 sec
ARIMA(1,0,2)(1,0,0)[12] intercept : AIC=inf, Time=0.54 sec
ARIMA(0,0,0)(1,0,0)[12] intercept : AIC=828.903, Time=0.04 sec
ARIMA(0,0,2)(1,0,0)[12] intercept : AIC=831.894, Time=0.06 sec
ARIMA(2,0,0)(1,0,0)[12] intercept : AIC=824.525, Time=0.09 sec
ARIMA(2,0,2)(1,0,0)[12] intercept : AIC=823.021, Time=0.38 sec
ARIMA(1,0,1)(1,0,0)[12] : AIC=820.901, Time=0.10 sec

Best model: ARIMA(1,0,1)(1,0,0)[12] intercept
Total fit time: 11.105 seconds

Best ARIMA Order: (1, 0, 1)
Best Seasonal Order: (1, 0, 0, 12)
Test RMSE for 2014 Prediction (Seasonal): 19267.74326738772

```

Seasonal ARIMA Backtesting: Train (2011-2013) vs Test (2014)



```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from pmdarima import auto_arima

# Step 1: Set your Date column as the dataframe index
# (Note: Agar aapke date column ka naam 'Order Date' ki jagah kuch aur ho, toh yahan change kar lena)
date_column_name = 'Order Date'

df[date_column_name] = pd.to_datetime(df[date_column_name])
df = df.set_index(date_column_name)
df = df.sort_index()

# Step 2: Clean Sales column and aggregate transactions into monthly totals
df['Sales'] = pd.to_numeric(df['Sales'], errors='coerce')
monthly_data = df['Sales'].resample('MS').sum().dropna()

print(f"Total monthly data points ready for training: {len(monthly_data)}")

# Step 3: Train the Seasonal ARIMA model (m=12 for monthly seasonality)
model = auto_arima(monthly_data,
                    seasonal=True,
                    m=12,
                    suppress_warnings=True,
                    error_action='ignore',
                    stepwise=True)

# Step 4: Generate the 12-month forecast for 2015

```

```

forecast_values = model.predict(n_periods=12)

# Step 5: Create the proper date index for 2015 (Month Start frequency)
future_dates = pd.date_range(start='2015-01-01', periods=12, freq='MS')

# Step 6: Create the final output DataFrame linking dates with predicted sales
forecast_df = pd.DataFrame({'Predicted_Sales': forecast_values}, index=future_dates)

print("\n 2015 Sales Forecast Table ")
display(forecast_df)

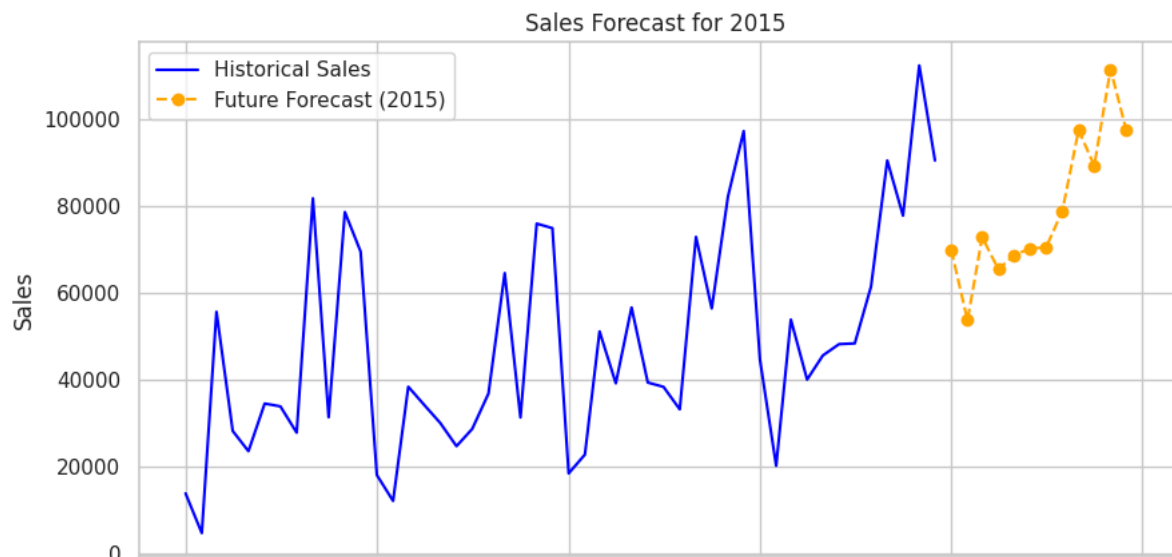
# Step 7: Plot the historical data and future forecast seamlessly
plt.figure(figsize=(10, 5))
plt.plot(monthly_data.index, monthly_data, label='Historical Sales', color='blue')
plt.plot(forecast_df.index, forecast_df['Predicted_Sales'], label='Future Forecast (2015)', color='orange', linestyle='--',
plt.title('Sales Forecast for 2015')
plt.xlabel('Timeline')
plt.ylabel('Sales')
plt.legend()
plt.grid(True)
plt.show()

```

Total monthly data points ready for training: 48

2015 Sales Forecast Table

	Predicted_Sales
2015-01-01	69941.880228
2015-02-01	53749.114299
2015-03-01	72860.146537
2015-04-01	65442.223841
2015-05-01	68739.832318
2015-06-01	70252.747130
2015-07-01	70501.458834
2015-08-01	78859.919519
2015-09-01	97449.231117
2015-10-01	89313.348651
2015-11-01	111472.926445
2015-12-01	97447.541942



```
!pip install streamlit plotly pandas scikit-learn statsmodels
```

```

Requirement already satisfied: streamlit in /usr/local/lib/python3.13/dist-packages (1.64.0)
Requirement already satisfied: plotly in /usr/local/lib/python3.13/dist-packages (5.24.1)
Requirement already satisfied: pandas in /usr/local/lib/python3.13/dist-packages (2.2.3)
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.13/dist-packages (1.6.1)
Requirement already satisfied: statsmodels in /usr/local/lib/python3.13/dist-packages (0.15.0)
Requirement already satisfied: altair!=5.4.0,!5.4.1,<7,>=5.0.0 in /usr/local/lib/python3.13/dist-packages (from streamlit)
Requirement already satisfied: click<9,>=7.0 in /usr/local/lib/python3.13/dist-packages (from streamlit) (8.5.0)
Requirement already satisfied: numpy<3,>=1.23 in /usr/local/lib/python3.13/dist-packages (from streamlit) (2.1.3)
Requirement already satisfied: packaging>=20 in /usr/local/lib/python3.13/dist-packages (from streamlit) (26.3)
Requirement already satisfied: pillow<13,>=7.1.0 in /usr/local/lib/python3.13/dist-packages (from streamlit) (11.3.0)
Requirement already satisfied: pydeck<1,>=0.8.0b4 in /usr/local/lib/python3.13/dist-packages (from streamlit) (0.9.3)
Requirement already satisfied: protobuf<8,>=5.26.1 in /usr/local/lib/python3.13/dist-packages (from streamlit) (5.29.6)

```

```
Requirement already satisfied: pyarrow!=25.0.0,<26,>=7.0 in /usr/local/lib/python3.13/dist-packages (from streamlit) (23.0.1)
Requirement already satisfied: requests<3,>=2.27 in /usr/local/lib/python3.13/dist-packages (from streamlit) (2.32.4)
Requirement already satisfied: toml<2,>=0.10.1 in /usr/local/lib/python3.13/dist-packages (from streamlit) (0.10.2)
Requirement already satisfied: typing-extensions<5,>=4.10.0 in /usr/local/lib/python3.13/dist-packages (from streamlit) (4.11.0)
Requirement already satisfied: starlette<2,>=0.46.0 in /usr/local/lib/python3.13/dist-packages (from streamlit) (1.6.0)
Requirement already satisfied: uvicorn<1,>=0.30.0 in /usr/local/lib/python3.13/dist-packages (from streamlit) (0.52.4)
Requirement already satisfied: httpxtools<1,>=0.6.3 in /usr/local/lib/python3.13/dist-packages (from streamlit) (0.8.0)
Requirement already satisfied: anyio<5,>=4.0.0 in /usr/local/lib/python3.13/dist-packages (from streamlit) (4.14.2)
Requirement already satisfied: python-multipart<1,>=0.0.10 in /usr/local/lib/python3.13/dist-packages (from streamlit) (0.0.19)
Requirement already satisfied: websockets<17,>=12.0.0 in /usr/local/lib/python3.13/dist-packages (from streamlit) (15.0.1)
Requirement already satisfied: itsdangerous<3,>=2.1.2 in /usr/local/lib/python3.13/dist-packages (from streamlit) (2.2.0)
Requirement already satisfied: watchdog<7,>=2.1.5 in /usr/local/lib/python3.13/dist-packages (from streamlit) (6.0.0)
Requirement already satisfied: tenacity>=6.2.0 in /usr/local/lib/python3.13/dist-packages (from plotly) (9.1.4)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.13/dist-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.13/dist-packages (from pandas) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.13/dist-packages (from pandas) (2026.3)
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.13/dist-packages (from scikit-learn) (1.16.3)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.13/dist-packages (from scikit-learn) (1.6.0)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.13/dist-packages (from scikit-learn) (3.6.0)
Requirement already satisfied: patsy>=0.5.6 in /usr/local/lib/python3.13/dist-packages (from statsmodels) (1.0.3)
Requirement already satisfied: formulaic>=1.1.0 in /usr/local/lib/python3.13/dist-packages (from statsmodels) (1.2.2)
Requirement already satisfied: Jinja2 in /usr/local/lib/python3.13/dist-packages (from altair!=5.4.0,!5.4.1,<7,>=5.0.0->streamlit) (3.1.4)
Requirement already satisfied: jsonschema>=3.0 in /usr/local/lib/python3.13/dist-packages (from altair!=5.4.0,!5.4.1,<7,>=5.0.0->streamlit) (4.19.0)
Requirement already satisfied: narwhals>=1.14.2 in /usr/local/lib/python3.13/dist-packages (from altair!=5.4.0,!5.4.1,<7,>=5.0.0->streamlit) (1.42.0)
Requirement already satisfied: idna>=2.8 in /usr/local/lib/python3.13/dist-packages (from anyio<5,>=4.0.0->streamlit) (3.19)
Requirement already satisfied: interface-meta>=1.2.0 in /usr/local/lib/python3.13/dist-packages (from formulaic>=1.1.0->statsmodels) (1.3.0)
Requirement already satisfied: wrapt>=1.17.0rc1 in /usr/local/lib/python3.13/dist-packages (from formulaic>=1.1.0->statsmodels) (1.17.0)
Requirement already satisfied: cloudpickle>=3.0 in /usr/local/lib/python3.13/dist-packages (from joblib>=1.2.0->scikit-learn) (3.1.1)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.13/dist-packages (from python-dateutil>=2.8.2->pandas) (1.17.0)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.13/dist-packages (from requests<3,>=2.27->streamlit) (3.4.0)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.13/dist-packages (from requests<3,>=2.27->streamlit) (2.3.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.13/dist-packages (from requests<3,>=2.27->streamlit) (2025.11.11)
Requirement already satisfied: h11>=0.8 in /usr/local/lib/python3.13/dist-packages (from uvicorn<1,>=0.30.0->streamlit) (0.14.0)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.13/dist-packages (from Jinja2->altair!=5.4.0,!5.4.1,<7,>=5.0.0->streamlit) (3.0.2)
Requirement already satisfied: attrs>=22.2.0 in /usr/local/lib/python3.13/dist-packages (from jsonschema>=3.0->altair!=5.4.0,!5.4.1,<7,>=5.0.0->streamlit) (25.1.0)
Requirement already satisfied: jsonschema-specifications>=2023.03.6 in /usr/local/lib/python3.13/dist-packages (from jsonschema>=3.0->altair!=5.4.0,!5.4.1,<7,>=5.0.0->streamlit) (2025.9.1)
Requirement already satisfied: referencing>=0.28.4 in /usr/local/lib/python3.13/dist-packages (from jsonschema>=3.0->altair!=5.4.0,!5.4.1,<7,>=5.0.0->streamlit) (0.36.0)
Requirement already satisfied: rpds-py>=0.25.0 in /usr/local/lib/python3.13/dist-packages (from jsonschema>=3.0->altair!=5.4.0,!5.4.1,<7,>=5.0.0->streamlit) (0.25.0)
```

```
%writefile app.py
import streamlit as st
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from pmdarima import auto_arima

# --- PAGE CONFIGURATION ---
st.set_page_config(
    page_title="Enterprise Retail Intelligence & Forecasting Dashboard",
    page_icon="📊",
    layout="wide"
)

# --- SIDEBAR: NAVIGATION & CONTROLS ---
st.sidebar.header("📊 Dashboard Navigation")
analysis_section = st.sidebar.radio(
    "Select Analytical Module:",
    [
        "📄 Executive Summary & 2015 Forecast",
        "📊 EDA, Correlation & Discount Impact",
        "🌐 Regional Performance & KPIs",
        "🎯 RFM & Customer Segmentation (K-Means/CLV)",
        "🔍 Advanced Diagnostics & PCA"
    ]
)

st.sidebar.markdown("---")
st.sidebar.info("**Capstone Project:** End-to-End Retail Sales Intelligence, Segmentation & Forecasting Framework.")

# --- DATA LOADING & CACHING ---
@st.cache_data
def load_and_prepare_data():
    df = pd.read_csv('/content/Superstore.csv', encoding='latin1')
    date_col = 'Order Date'
    df[date_col] = pd.to_datetime(df[date_col], dayfirst=True)
    df = df.sort_values(by=date_col)

    # Clean numeric fields
    df['Sales'] = pd.to_numeric(df['Sales'], errors='coerce')
    df['Profit'] = pd.to_numeric(df['Profit'], errors='coerce')
    df['Discount'] = pd.to_numeric(df['Discount'], errors='coerce')

    # Monthly Aggregation for Forecasting
    monthly_data = df.set_index(date_col)['Sales'].resample('MS').sum().dropna()
```

```

return df, monthly_data

with st.spinner('Loading integrated datasets and computing analytics...'):
    df, monthly_data = load_and_prepare_data()

# Train ARIMA model for forecasting section
@st.cache_resource
def get_forecast_model(data):
    model = auto_arima(data, seasonal=True, m=12, suppress_warnings=True, error_action='ignore', stepwise=True)
    preds = model.predict(n_periods=12)
    future_idx = pd.date_range(start='2015-01-01', periods=12, freq='MS')
    return pd.DataFrame({'Predicted_Sales': preds}, index=future_idx), model

forecast_df, arima_model = get_forecast_model(monthly_data)

# --- MODULE 1: EXECUTIVE SUMMARY & FORECAST ---
if analysis_section == "📊 Executive Summary & 2015 Forecast":
    st.title("📊 Executive Summary & 2015 Sales Forecast")
    st.markdown("High-level overview of historical business scale combined with automated Seasonal ARIMA revenue predicti")
    st.markdown("---")

    # Top-line metrics
    c1, c2, c3, c4 = st.columns(4)
    c1.metric("Total Historical Revenue", f"${df['Sales'].sum():.2f}")
    c2.metric("Total Transactions", f"{len(df):,}")
    c3.metric("2015 Projected Sales", f"${forecast_df['Predicted_Sales'].sum():.2f}")
    c4.metric("Peak Forecast Month", forecast_df['Predicted_Sales'].idxmax().strftime('%B %Y'))

    st.markdown("---")

    col_l, col_r = st.columns([2, 1])
    with col_l:
        st.subheader("Historical Trajectory vs 2015 Forecast Curve")
        fig, ax = plt.subplots(figsize=(10, 5))
        ax.plot(monthly_data.index, monthly_data, label='Historical Sales (2011-2014)', color='#1f77b4', linewidth=2)
        ax.plot(forecast_df.index, forecast_df['Predicted_Sales'], label='2015 Forecast', color='#ff7f0e', linestyle='--')
        ax.set_title("Seasonal ARIMA Trend Analysis")
        ax.set_xlabel("Timeline")
        ax.set_ylabel("Sales ($)")
        ax.legend()
        ax.grid(True, linestyle=':', alpha=0.6)
        st.pyplot(fig)

    with col_r:
        st.subheader("2015 Monthly Breakdown")
        display_tbl = forecast_df.copy()
        display_tbl['Predicted_Sales'] = display_tbl['Predicted_Sales'].map('${:,.2f}'.format)
        st.dataframe(display_tbl, height=310)

# --- MODULE 2: EDA, CORRELATION & DISCOUNT IMPACT ---
elif analysis_section == "📈 EDA, Correlation & Discount Impact":
    st.title("📈 Exploratory Data Analysis & Profitability Drivers")
    st.markdown("In-depth analysis of feature distributions, logarithmic transformations for skewness, and the direct imp")
    st.markdown("---")

    col1, col2 = st.columns(2)

    with col1:
        st.subheader("Impact of Discount on Profitability")
        st.write("Examining how higher discount margins severely compress profit margins across orders.")
        fig, ax = plt.subplots(figsize=(7, 4))
        ax.scatter(df['Discount'], df['Profit'], alpha=0.4, color='#d62728')
        ax.set_title("Discount vs. Profit Relationship")
        ax.set_xlabel("Discount Rate")
        ax.set_ylabel("Profit ($)")
        ax.grid(True, linestyle=':', alpha=0.5)
        st.pyplot(fig)

    with col2:
        st.subheader("Log-Transformation of Sales (Skewness Reduction)")
        st.write("Normalizing heavy-tailed transactional data distribution using log scaling.")
        fig, ax = plt.subplots(figsize=(7, 4))
        log_sales = np.log1p(df['Sales'].dropna())
        ax.hist(log_sales, bins=30, color='#2ca02c', edgecolor='black', alpha=0.7)
        ax.set_title("Log-Transformed Sales Distribution")
        ax.set_xlabel("Log(Sales + 1)")
        ax.set_ylabel("Frequency")
        st.pyplot(fig)

    st.markdown("---")
    st.subheader("Key Statistical Insights & Correlation Table")
    st.write("Correlation matrix insights reflecting structural feature associations:")

```

```

numeric_df = df[['Sales', 'Profit', 'Discount']].dropna()
st.dataframe(numeric_df.corr().style.background_gradient(cmap='coolwarm'), use_container_width=True)

# --- MODULE 3: REGIONAL PERFORMANCE & KPIS ---
elif analysis_section == "🌐 Regional Performance & KPIS":
    st.title("🌐 Regional Performance & Market Share Analysis")
    st.markdown("Geographical breakdown of sales, profit distributions, and regional operational efficiencies.")
    st.markdown("---")

    if 'Region' in df.columns:
        reg_summary = df.groupby('Region')[['Sales', 'Profit']].sum().reset_index()

        c1, c2 = st.columns(2)
        with c1:
            st.subheader("Total Sales by Region")
            fig, ax = plt.subplots(figsize=(7, 4))
            ax.bar(reg_summary['Region'], reg_summary['Sales'], color='#1f77b4')
            ax.set_ylabel("Total Sales ($)")
            st.pyplot(fig)

        with c2:
            st.subheader("Total Profitability by Region")
            fig, ax = plt.subplots(figsize=(7, 4))
            ax.bar(reg_summary['Region'], reg_summary['Profit'], color='#2ca02c')
            ax.set_ylabel("Total Profit ($)")
            st.pyplot(fig)

        st.markdown("---")
        st.subheader("Detailed Regional Performance Metrics Table")
        reg_summary['Profit Margin (%)'] = (reg_summary['Profit'] / reg_summary['Sales']) * 100
        st.dataframe(reg_summary.style.format({'Sales': '${:,.2f}', 'Profit': '${:,.2f}', 'Profit Margin (%)': '{:.2f}%'})

```